

Engenharia de Software

Processos

Prof. Tiago Piperno Bonetti

Material adaptado de: Marco Tulio Valente
<https://engsoftmoderna.info>

Importância de Processos

A produção de um **carro** em uma fábrica de automóveis **segue** um **processo** bem definido.

- » Primeiro, as chapas de aço são cortadas e prensadas, para ganhar a forma de portas, tetos e capôs. Depois, o carro é pintado e instalam-se painel, bancos, cintos de segurança e a fiação. Por fim, instala-se a parte mecânica, incluindo motor, suspensão e freios.

Software também é produzido de acordo com um **processo**.

- » Um processo de desenvolvimento de software define um conjunto de passos, tarefas, eventos e práticas que devem ser seguidos por desenvolvedores de software, na produção de um sistema.

Importância de Processos

Na prática, os sistemas modernos — que nos interessam— são desenvolvidos em **equipes**.

E essas equipes precisam de um **ordenamento**, mesmo que mínimo.

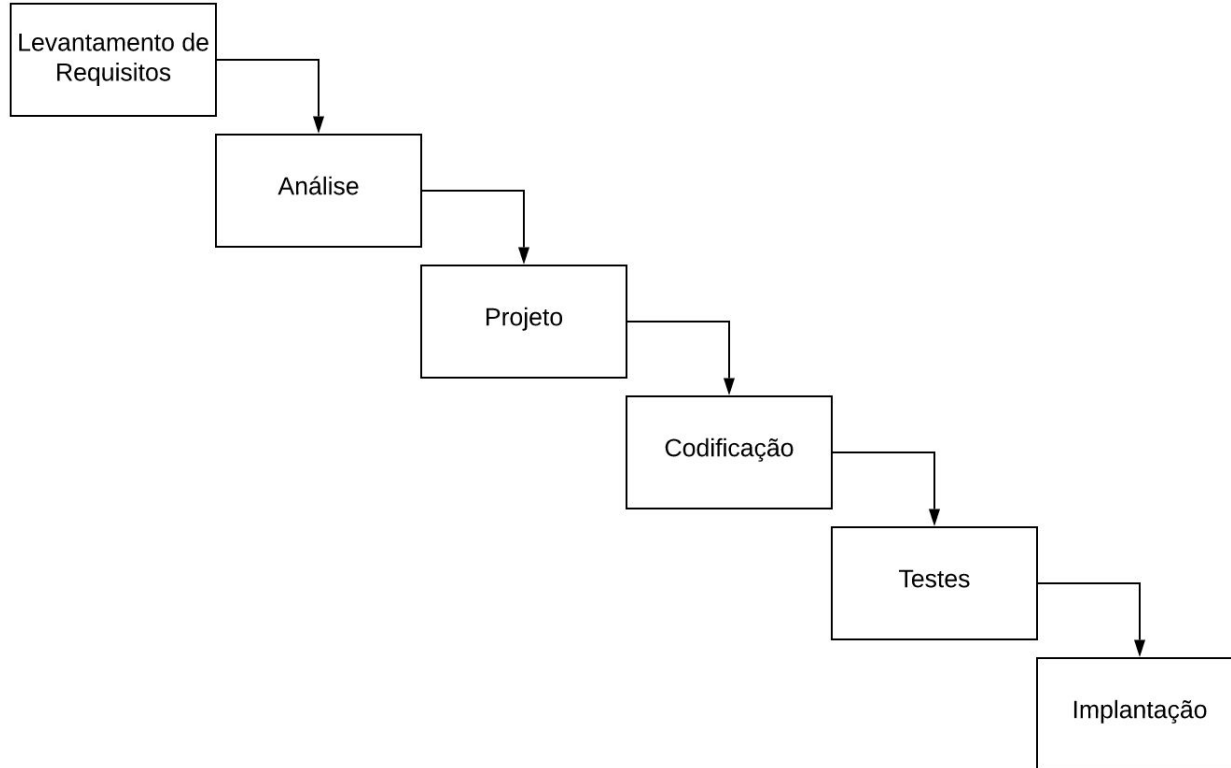
Processos são o **instrumento** de que as empresas dispõem para **coordenar**, **motivar**, **organizar** e **avaliar** o **trabalho** de seus desenvolvedores.

Processos também são importantes para os **desenvolvedores**, pois permitem que eles tomem consciência das **tarefas** e **resultados** que se **esperam deles**.

Engenharia Tradicional

- » Civil, mecânica, elétrica, aviação, automobilística, etc
- » Projeto com duas características:
 - **Planejamento detalhado**
 - Sequencial
- » Isto é: Waterfall
 - Sequenciais e precedidos de um planejamento detalhado

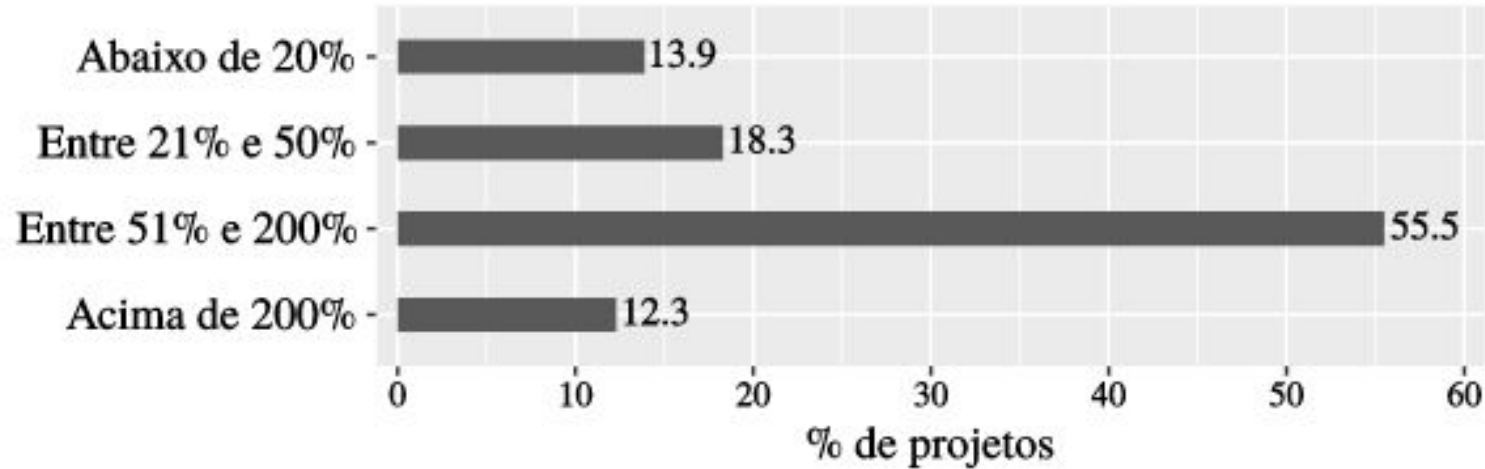
Natural que ES começasse usando Waterfall



Engenharia Tradicional

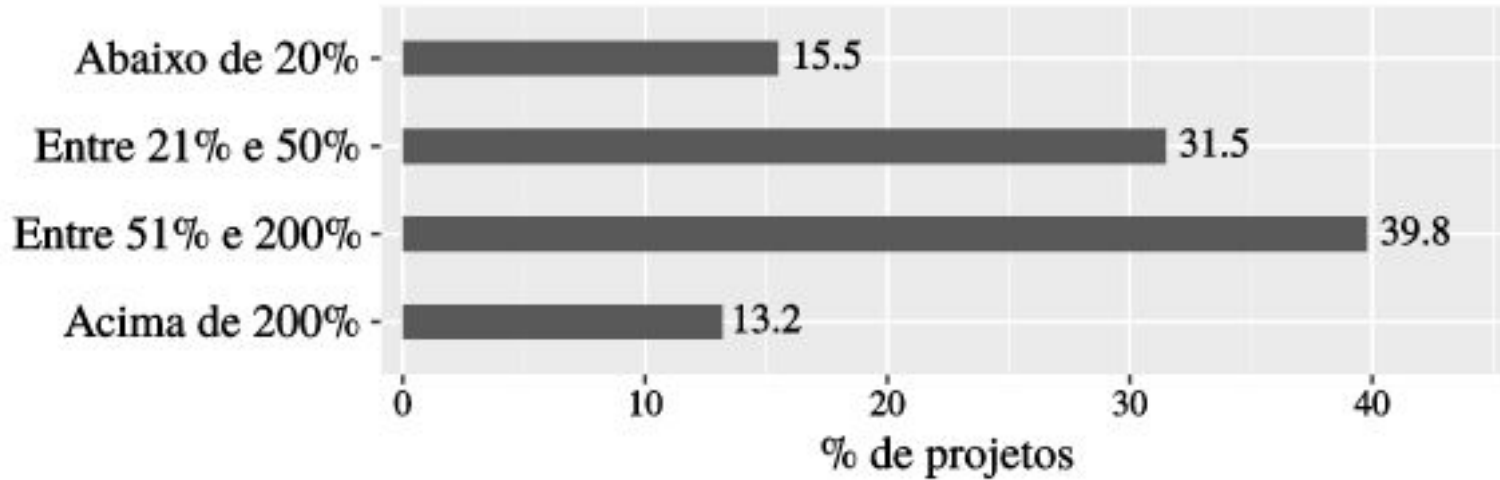
- » No entanto, começou-se a perceber que software é **diferente** de outros produtos de Engenharia
 - Os cronogramas e orçamentos desses projetos não eram obedecidos.
 - Projetos inteiros eram cancelados, após anos de trabalho, sem entregar um sistema funcional para os clientes.
- » Em 1994, um **relatório** produzido pela empresa de consultoria Standish Group revelou **informações** mais **detalhadas** sobre os **projetos** de **software** da época.

Engenharia Tradicional



CHAOS Report (1994): percentual de projetos que estourava seus **prazos** (para cada faixa de estouro).

Engenharia Tradicional



CHAOS Report (1994): percentual de projetos que estourava seus **orçamentos** (para cada faixa de estouro).

Dificuldade 1: Requisitos

- » Clientes não sabem o que querem (em um software)
 - Funcionalidades são "infinitas" (difícil prever)
 - Mundo muda!
- » Não dá mais para ficar 1 ano levantando requisitos, 1 ano projetando, 1 ano implementando, etc
- » Quando o software ficar pronto, ele estará obsoleto!

Dificuldade 2: Documentações Detalhadas

- » Verbosas e pouco úteis
- » Na prática, desconsideradas durante implementação
- » *Plan-and-document* não funcionou com software



Manifesto Ágil

Em 2001, um grupo de profissionais da indústria se reuniu na cidade de Snowbird, no estado norte-americano de Utah, para discutir e propor uma alternativa aos processos do tipo Waterfall que então predominavam.

Essencialmente, eles passaram a defender que software é diferente de produtos tradicionais de Engenharia.

Por isso, software também demanda um processo de desenvolvimento diferente.



Manifesto Ágil

Por meio deste trabalho, passamos a valorizar:

Indivíduos e interações, mais do que processos e ferramentas

Software em funcionamento, mais do que documentação abrangente

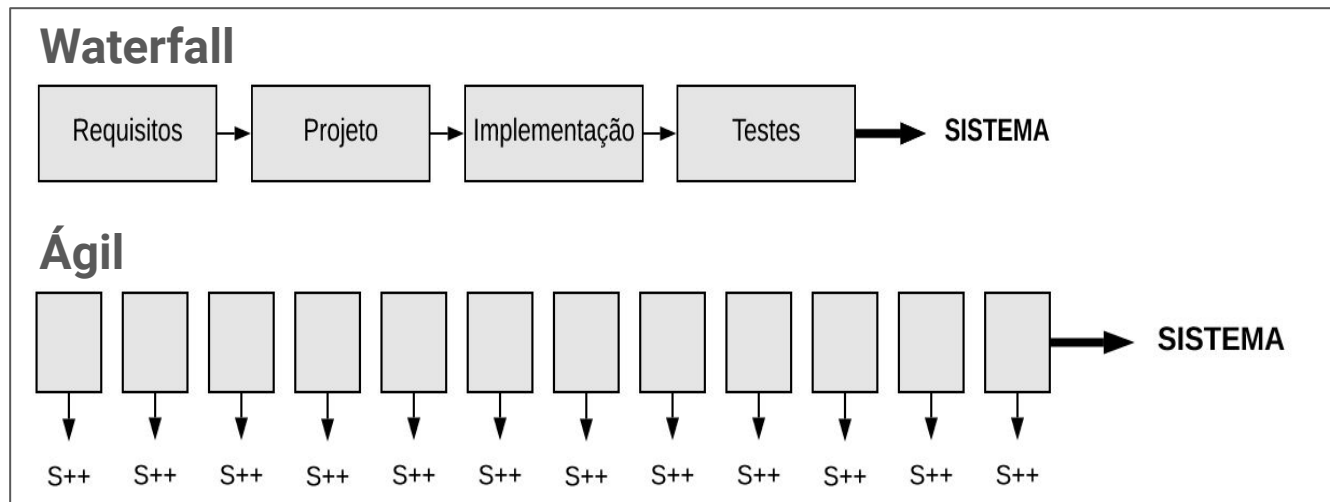
Colaboração com o cliente, mais do que negociação de contratos

Resposta a mudanças, mais do que seguir um plano.

Manifesto Ágil

A característica principal de processos ágeis é a adoção de **ciclos curtos e iterativos de desenvolvimento**.

Um sistema é implementado de forma gradativa; começando por aquilo que é mais urgente.



Desenvolvimento usando um Processo Ágil. A cada iteração (representada pelos retângulos) gera-se um incremento no sistema (S++), que já pode ser validado e testado pelos usuários finais

Desenvolvimento iterativo

Implementa-se uma **primeira versão** do sistema, com as **funcionalidades** que possuem **prioridade máxima**.

Em seguida, essa versão é **validada** pelo cliente.

» Se ela for aprovada, um novo ciclo — ou **iteração** — inicia-se.

Normalmente, esses **ciclos** são **curtos**, com duração de um mês, talvez até um pouco menos.

O sistema vai sendo **construído** de forma **incremental**

» Cada incremento devidamente aprovado pelos clientes.

O desenvolvimento termina quando o **cliente** decide que **todos** os **requisitos** estão **implementados**.

Desenvolvimento iterativo

- » Suponha um sistema imenso, complexo etc
- » Qual o menor "incremento de sistema" eu consigo implementar em 15 dias e validar com o usuário?
- » Validar é muito importante!
- » Cliente não sabe o que quer!

Pontos importantes

Menor ênfase em documentação

- » Apenas o essencial deve ser documentado.

Menor ênfase em planos detalhados

- » Muitas vezes nem o cliente, nem os Engenheiros de Software têm (no início) uma ideia clara dos requisitos.
- » Esse entendimento vai surgir à medida que os incrementos de produto sejam produzidos e validados.
- » O importante é conseguir avancar, mesmo em ambientes com informações imperfeitas, parciais e sujeitas a mudanças.

Pontos importantes

Inexistência de uma fase dedicada a design.

- » O design também é incremental. Ele evolui à medida que o sistema vai nascendo, ao final de cada iteração.

Desenvolvimento em times pequenos

- » Times que possam ser alimentados com duas pizzas, conforme popularizado pelo CEO da Amazon, Jeff Bezos.

Ênfase em novas práticas de desenvolvimento

- » Como programação em pares, testes automatizados e integração contínua.

Métodos Ágeis



Métodos Ágeis

- » Alguns métodos foram propostos para **ajudar** desenvolvedores a **adotar** os **princípios ágeis**
- » Dão mais **consistência** às ideias ágeis
 - Definem um processo, mesmo que leve
 - Workflow, eventos, papéis, práticas, princípios etc

Métodos Ágeis (exemplos)

Extreme Programming (XP)

- » Proposto por Kent Beck, em um livro lançado em 1999 ([link](#)). Uma segunda edição do livro, incluindo uma grande revisão, foi lançada em 2004.

Scrum

- » Proposto por Jeffrey Sutherland e Ken Schwaber, em um artigo publicado em 1995 ([link](#)).

Kanban

- » Origens remontam a um sistema de controle de produção que começou a ser usado nas fábricas da Toyota, ainda na década de 50 ([link](#)). Nos últimos 10 anos, Kanban tem sido gradativamente adaptado para uso no desenvolvimento de software.

Antes de começar

- » Nenhum processo é uma **bala-de-prata**
- » Processo ajuda a **não cometer** certos "grandes erros"
- » Processos **não** são adotados **100% igual** ao manual
 - Bom senso é importante
 - Experimentação é importante

Mundo Real

Hoje, a grande **maioria** das **empresas** usam **princípios ágeis**, em maior ou menor escala.

Em 2018, o Stack Overflow survey incluiu uma pergunta sobre o método de desenvolvimento mais usado pelos respondentes ([link](#)).

- » A grande **maioria** mencionou métodos ou práticas ágeis, como **Scrum** (63% das respostas), **Kanban** (36%) e **Extreme Programming** (16%).
- » Apenas **15%** dos participantes marcaram **Waterfall** como resposta.

Software é um trabalho em equipe (daí a importância de processos)

Luis André Barroso, brasileiro que é VP de Engenharia do Google (vídeo ~1.5 min)

https://youtu.be/S7A6SYI_nbc



Extreme Programming (XP)



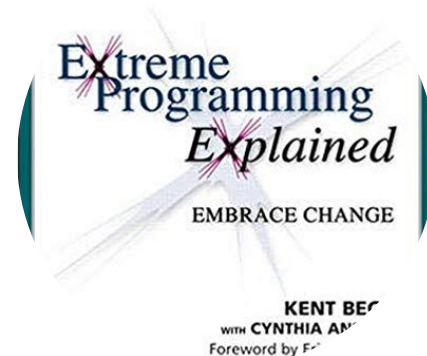
Extreme Programming



Kent Beck



1999



2004

Extreme Programming

É um método **leve** recomendado para desenvolver software com **requisitos vagos** ou sujeitos a **mudanças**;

XP possui todas as características dos métodos ágeis

- » **Ciclos** curtos e iterativos de desenvolvimento;
- » Menos ênfase para **documentação** e para planos detalhados;
- » Propõe que o **design** de um sistema também seja definido de forma **incremental**;
- » Sugere que as **equipes** de desenvolvimento sejam pequenas.

Extreme Programming

Não é um método **prescritivo**, que define um passo a passo detalhado para construção de software.

É definido por meio de um conjunto de **valores, princípios e práticas de desenvolvimento**.

Inicialmente definido de forma **abstrata**, usando-se de valores e princípios que devem fazer parte da **cultura e dos hábitos** de times de desenvolvimento de software.

Extreme Programming

XP =
Valores + Princípios + Práticas

XP - Valores

O desenvolvimento de projetos de software deve ser norteado por **três** valores **principais**:

Comunicação

» É importante em qualquer projeto, para **evitar** e **aprender** com **erros**.

Simplicidade

» Em todo sistema complexo e desafiador existem **sistemas** ou **subsistemas** mais **simples**, que às vezes não são considerados.

XP - Valores

Feedback

- » Existem **riscos** em todos os projetos: os **requisitos** mudam, a **tecnologia** muda, a **equipe** de desenvolvimento muda, o **mundo** muda, etc.
- » Um valor que ajuda a **controlar** tais **riscos** é estar **aberto ao feedback** dos **stakeholders**, a fim de que correções de rota sejam implementadas o quanto antes.
- » Em outras palavras, é **difícil** desenvolver o sistema de software **certo** em uma **primeira e única tentativa**.

XP - Valores

Feedback

» Frederick Brooks tem uma frase conhecida sobre esse fenômeno:

*"Planeje-se para jogar fora partes de seu sistema,
pois você fará isso."*

XP - Valores

XP também defende **outros valores**, como:

- » Coragem;
- » Respeito;
- » Qualidade de vida.

Extreme Programming

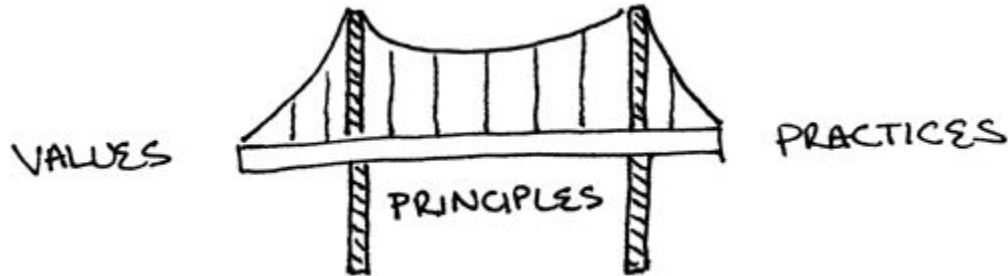
XP =
Valores + Princípios + Práticas

XP - Princípios

Os valores são abstratos e universais.

As práticas são procedimentos concretos e pragmáticos.

Para **unir** esses **dois extremos**, XP defende que projetos de software devem seguir um conjunto de princípios.



XP - Princípios

Humanidade (*humanity*)

Software é uma atividade **intensiva** no uso de **capital humano**.

O **principal recurso** de uma empresa de software não são seus bens físicos, mas sim seus **colaboradores**.

Um termo que reflete bem esse princípio é **peopleware** (*foi cunhado por Tom DeMarco, em um livro com o mesmo título (link)*).

A ideia é que a **gestão de pessoas** — incluindo fatores como expectativas, crescimento, motivação, transparência e responsabilidade — é fundamental para o sucesso de projetos de software.

XP - Princípios

Economicidade (*economics*)

Software é uma **atividade cara**, que demanda a alocação de recursos financeiros consideráveis.

Quem está pagando as contas do projeto, espera **resultados econômicos e financeiros**.

Software não é uma obra de arte.

XP - Princípios

Benefícios Mútuos

As decisões tomadas em um projeto de software têm que **beneficiar múltiplos stakeholders**.

Exemplo: o **contratante** do software deve garantir um bom **ambiente de trabalho** (peopleware); em contrapartida, a **equipe** deve entregar um **sistema** que **agregue valor** ao seu negócio (economicidade).

A frase todo negócio tem que ser bom para os dois lados resume bem esse terceiro princípio de XP.

XP - Princípios

Melhorias Contínuas *(no livro de XP, o nome original é improvements)*

Nenhum **processo** de desenvolvimento de software é **perfeito**.

É mais **seguro** trabalhar com um **sistema** que vai sendo **continuamente aprimorado**, a cada iteração, com o feedback dos clientes e de todos os membros do time.

Pelo mesmo motivo, XP não recomenda investir um grande montante de **tempo** em um **design inicial e completo**.

As **próprias práticas de desenvolvimento** podem ser aprimoradas; para isso, o time deve reservar tempo para refletir sobre elas.

XP - Princípios

Falhas Acontecem

Software é uma das **mais complexas** construções humanas.

Falhas **incluem** bugs, funcionalidades que não se mostraram interessantes para os usuários e requisitos não-funcionais que não estão sendo atendidos, como desempenho, usabilidade, privacidade etc.

Elas **não** devem ser usadas para **punir membros** de um time.

Falhas **fazem parte** do **jogo**, se um time pretende entregar software com **rapidez**.

XP - Princípios

Baby Steps *(Passos de bebê)*

É melhor um **progresso seguro, testado e validado**, mesmo que pequeno, do que grandes implementações com riscos de serem **descartadas** pelos usuários.

O importante é garantir melhorias contínuas, não importando que sejam pequenas, desde que na **direção correta**.

Essas pequenas melhorias são **melhores** do que **grandes revoluções**, as quais costumam não apresentar resultados positivos, pelo menos quando se trata de desenvolvimento de software.

XP - Princípios

Responsabilidade Pessoal *(accepted responsibility)*

Desenvolvedores devem ter uma **ideia clara** de seu **papel** e **responsabilidade** na equipe.

O motivo é que responsabilidade não pode ser transferida, sem que a **outra parte a aceite**.

XP defende que o engenheiro de software que implementa uma história — termo que o método usa para requisitos — deve ser também aquele que vai testá-la e mantê-la.